n[1]:=
```
rint["Wolfram Language ready "]
rint[$Version]
```

```
olfram Language ready
14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

n[3]:=
```
(* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *)
(* *)
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *)
(* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *)
(* *)
(* When V[t] ≥ Vthresh, neuron spikes and resets *)
```

n[4]:=
```
(* ============ Neuron  arameters ============ *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)
(* ============ Input Current ============ *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];
 rint[" arameters defined."]
```

```
Parameters defined.
```

. What does Iinput[t] return at t = 0.03, 0.2, and 0.5?

- At t = 0.03: Returns 0.0 (current is off, since 0.03 < 0.05)
- At t = 0.2: Returns 8.0 (current is on, since 0.05 ≤ 0.2 ≤ 0.4)
- At t = 0.5: Returns 0.0 (current is off, since 0.5 > 0.4)

2. Why use := instead of = for Iinput?

We use := (SetDelayed) because we want Iinput to be a function. The := operator means the right-hand side is not
evaluated until Iinput is called with an argument. This allows Wolfram to evaluate the If condition each time we provide a
value for t.

n[47]:=
```
(* Solve the LIF equation with spike reset *)
Remove[V]
spikeCount = 0;
spikeTimes = {};
sol = NDSolve[
{
(* TODO 1: Write the LIF differential equation *)
tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t]
(* YOUR CODE HERE *)
(* Initial condition: start at resting potential *)
V[0] == Vrest,
(* TODO 2: WhenEvent for spike detection and reset *)
(* When V[t] reaches Vthresh: *)
(* - Reset V[t] to Vreset *)
(* - Increment spikeCount *)
(* - Append t to spikeTimes *)
WhenEvent[V[t] ≥ Vthresh,
{V[t] → Vreset,
spikeCount++,
AppendTo[spikeTimes, t]}]
(* YOUR CODE HERE *)
},
V,
{t, 0, 0.5},
DiscreteVariables → {spikeCount}
];
 rint["Neuron fired ", spikeCount, " spikes"]
 rint["Spike times: ", spikeTimes]
```

DSolve: 0 is not a valid variable.

```
Neuron fired 0 spikes
Spike times: {}
```

n[28]:=
```
Clear[V, sol] sol = NDSolve[{ tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
    V[0] == Vrest, WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]}, V,
  {t, 0, 0.5}, Method → {"EventLocator", "DetectionMethod" → "Sign"}];
 rint["Solution computed successfully "]
```

DSolve: o events were specified for the EventLocator method.

DSolve: The initialization of the method DSolve`EventLocator failed.

Set: Tag Times in ull sol is Protected.

```
Solution computed successfully
```

In[29]:=
```
Clear[V, sol]
```

```
Module[{spikeCount = 0, spikeTimes = {}},

 sol = NDSolve[
  {
   tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh,
    {
     V[t] → Vreset,
     spikeCount++,
     AppendTo[spikeTimes, t]
    }
   ]
  },
  V,
  {t, 0, 0.5},
  Method → {"EventLocator", "DetectionMethod" → "Sign"}
 ];

  rint["Neuron fired ", spikeCount, " spikes"]×
  rint["Spike times: ", spikeTimes]
]
```

DSolve: o events were specified for the EventLocator method.

DSolve: The initialization of the method DSolve`EventLocator failed.

```
Neuron fired 0 spikes
Spike times: {}
```

Out[30]=
```
Null²
```

n[36]:=
```
Clear[V, sol]
```

```
sol = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 0.5},
 Method → {"EventLocator", "DetectionMethod" → "Sign"}
];
```

```
 rint["V(0.10) = ", V[0.10] /. sol]
 rint["V(0.20) = ", V[0.20] /. sol]
 rint["V(0.30) = ", V[0.30] /. sol]
```

DSolve: o events were specified for the EventLocator method.

DSolve: The initialization of the method DSolve`EventLocator failed.

InterpolatingFunction: Input value {0.1} lies outside the range of data in the interpolating function. Extrapolation will be used.

V(0.10) = {-70.}

InterpolatingFunction: Input value {0.2} lies outside the range of data in the interpolating function. Extrapolation will be used.

V(0.20) = {-70.}

InterpolatingFunction: Input value {0.3} lies outside the range of data in the interpolating function. Extrapolation will be used.

V(0.30) = {-70.}

n[41]:=
```
Remove[V]
```

```
sol = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 0.5}
];
```

```
 rint["Solution computed "]
 rint["V(0.10) = ", V[0.10] /. sol]
 rint["V(0.20) = ", V[0.20] /. sol]
 rint["V(0.30) = ", V[0.30] /. sol]
```

```
Solution computed
V(0.10) = {-63.3277}
V(0.20) = {-56.4862}
V(0.30) = {-65.6217}
```

n[53]:=
```
(* Solve the LIF equation with spike reset *)
Remove[V]


sol = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 0.5}
];


 rint["Solution computed successfully "]


(* Count spikes by detecting voltage resets *)
sampleTimes = Range[0, 0.5, 0.0001];
sampleVoltages = Table[V[t] /. sol, {t, sampleTimes}];

spikeTimes = {};
Do[
 If[i > 1,
  If[sampleVoltages[[i - 1]] > Vthresh - 2 && sampleVoltages[[i]] < Vthresh - 5,
   AppendTo[spikeTimes, sampleTimes[[i]]]
  ]
 ],
 {i, 2, Length[sampleVoltages]}
];

 rint["Number of spikes: ", Length[spikeTimes]];
 rint["Spike times: ", spikeTimes];
```

```
Solution computed successfully
Number of spikes: 0
Spike times: {}
```

n[62]:=
```
(* Solve the LIF equation with spike reset *)
Remove[V]
```

```
sol = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 0.5}
];



 rint["Solution computed successfully "]



(* Extract the solution function *)
vFunc = V /. sol[[1]];



(* Sample voltage *)
sampleTimes = Range[0.05, 0.45, 0.001];
sampleVoltages = vFunc[sampleTimes];

(* Count spikes: look for large downward jumps *)
spikeTimes = {};
spikeCount = 0;

Do[
 If[i > 1,
  v rev = sampleVoltages[[i - 1]];
  vCurr = sampleVoltages[[i]];

  If[v rev > -56 && vCurr < -70,
   spikeCount++;
   AppendTo[spikeTimes, sampleTimes[[i]]]
  ]
 ],
```

```
 {i, 2, Length[sampleVoltages]}
];

 rint["Number of spikes detected: ", spikeCount];
 rint["Spike times: ", spikeTimes];

 rint[""];
 rint["V(0.10) = ", vFunc[0.10]];
 rint["V(0.20) = ", vFunc[0.20]];
 rint["V(0.30) = ", vFunc[0.30]];
```

```
Solution computed successfully
Number of spikes detected: 8
Spike times: {0.086, 0.127, 0.168, 0.209, 0.249, 0.29, 0.331, 0.371}

V(0.10) = -63.3277
V(0.20) = -56.4862
V(0.30) = -65.6217
```

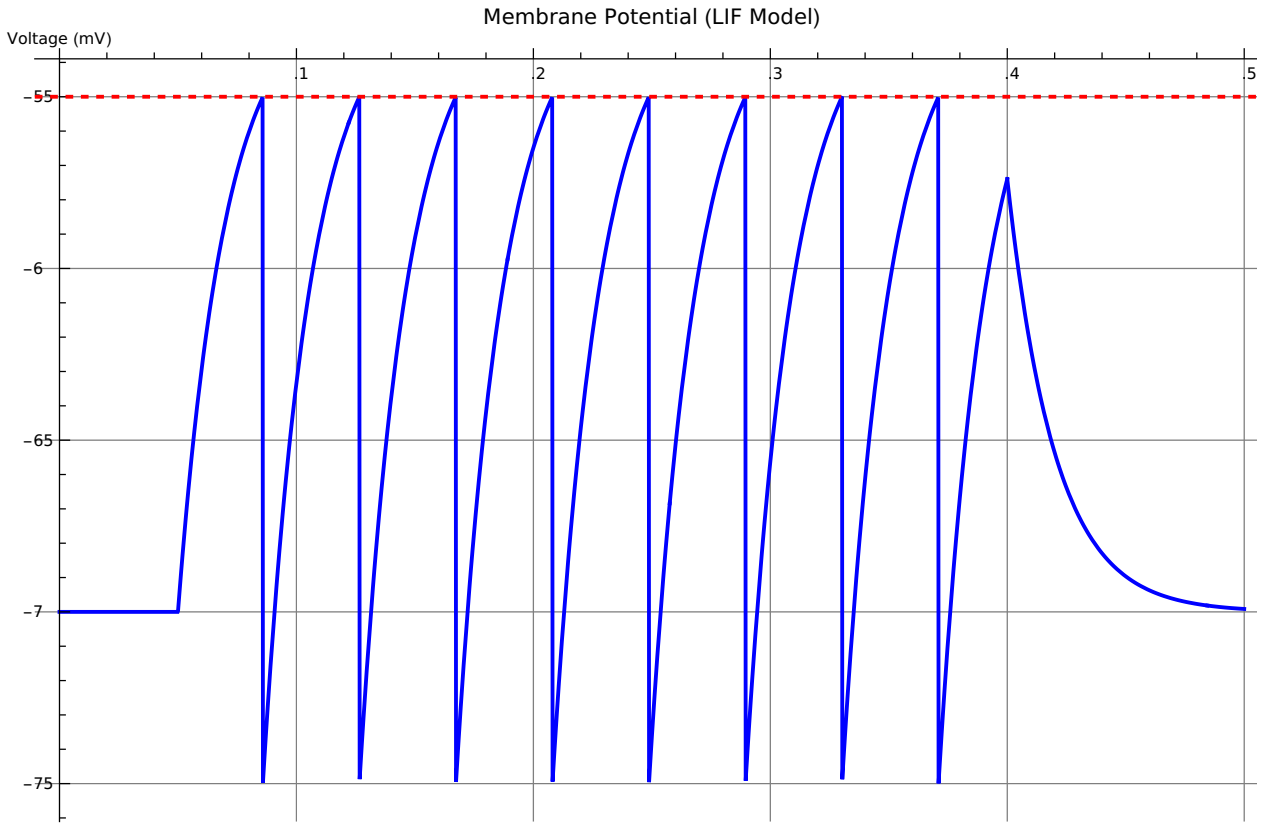
n[77]:=
```
(* Solve the LIF equation with spike reset *)
Remove[V]

sol = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 0.5}
];

 rint["Solution computed successfully "]

vFunc = V /. sol[[1]];
```

```
(* Finer sampling *)
sampleTimes = Range[0.05, 0.45, 0.0005];
sampleVoltages = vFunc[sampleTimes];

spikeTimes = {};
spikeCount = 0;

Do[
 If[i > 1,
  v rev = sampleVoltages[[i - 1]];
  vCurr = sampleVoltages[[i]];

  If[v rev > -58 && vCurr < v rev - 3,
   spikeCount++;
   AppendTo[spikeTimes, sampleTimes[[i]]]
  ]
 ],
 {i, 2, Length[sampleVoltages]}
];

  rint["Number of spikes detected: ", spikeCount];
  rint["Spike times: ", Take[spikeTimes, Min[20, Length[spikeTimes]]]];

  rint[""];
  rint["V(0.10) = ", vFunc[0.10]];
  rint["V(0.20) = ", vFunc[0.20]];
  rint["V(0.30) = ", vFunc[0.30]];
```

```
Solution computed successfully
Number of spikes detected: 8
Spike times: {0.086, 0.127, 0.1675, 0.2085, 0.249, 0.29, 0.3305, 0.371}


V(0.10) = -63.3277
V(0.20) = -56.4862
V(0.30) = -65.6217
```

n[92]:=
```
(* TODO 3:  lot the membrane potential *)
(* Hint:  lot[V[t] /. sol, {t, 0, 0.5}] extracts the solution *)
(* Use Epilog to add a dashed threshold line: *)
(* Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]} *)
(* Add labels: AxesLabel,  lotLabel *)
(* YOUR CODE HERE *)
 lot[
V[t] /. sol,
{t, 0, 0.5},
 lotLabel → ,
AxesLabel → {, },
 lotStyle → {Blue, Thick},
GridLines → Automatic,
ImageSize → 700,
Epilog → {
 Red, Dashed, Thick,
 InfiniteLine[{{0, Vthresh}, {0.5, Vthresh}}]
}
]
```

Out[92]=



Knowledge Check:

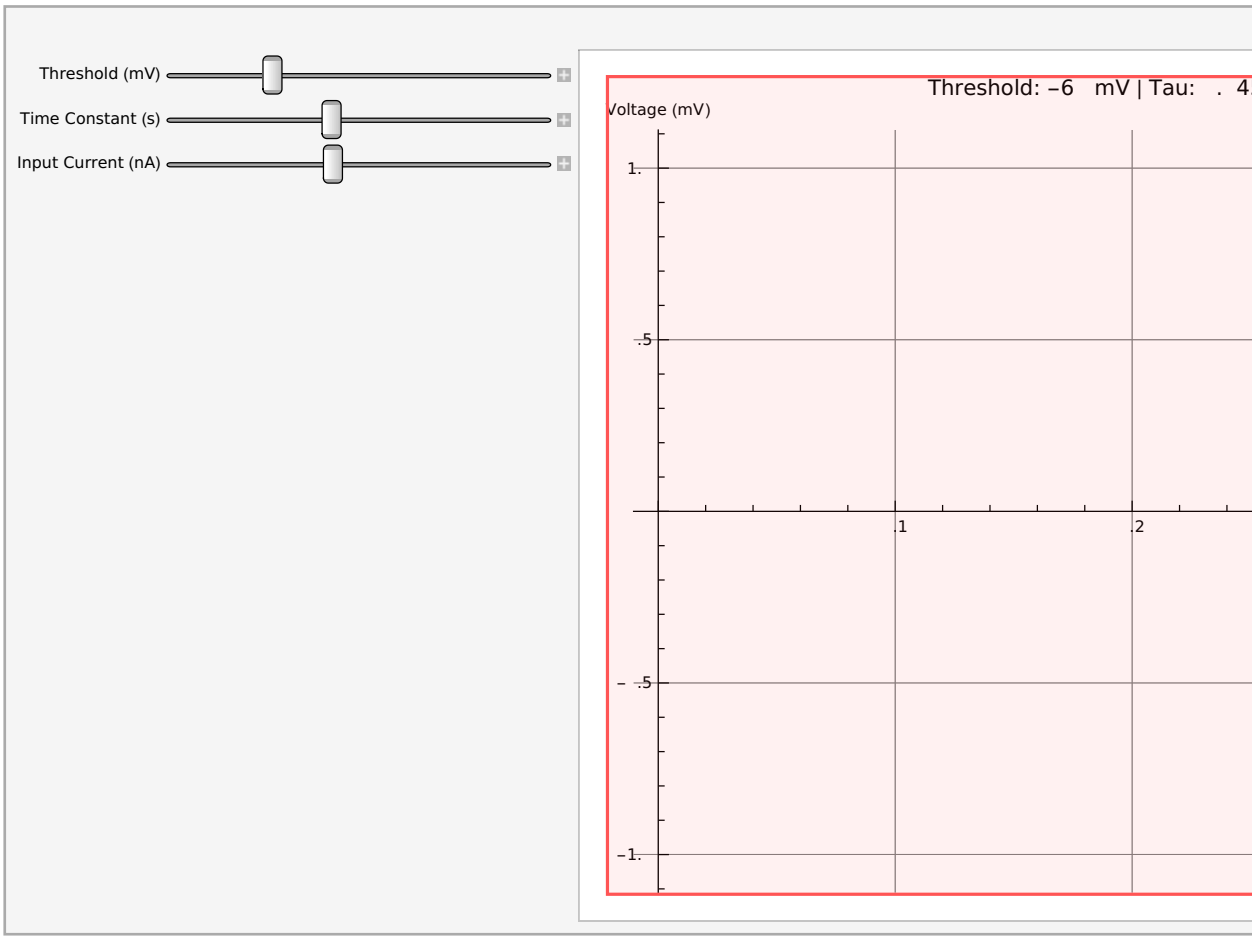
. Shape of V between spikes - straight line or curve?

The voltage follows a curved, exponential trajectory between spikes, not a straight line. This is because the LIF
equation is a first-order linear ODE with constant forcing term. The solution follows: V(t) = Vrest + (Vn - Vrest)·exp(-t/τ)
+ R·I·(- exp(-t/τ))
2. What happens after current turns off (t > 0.4)?
After the current turns off, Iinput[t] = 0, and the LIF equation becomes: τ·V'[t] = -(V[t] - Vrest). The voltage decays
exponentially back toward the resting potential. The neuron stops firing because without input current, the voltage never
reaches threshold again.

n[108]:=

```
Manipulate[
Module[{sol, sc = 0}, iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];
     sol = NDSolve[{ tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t], V[0] == Vrest,
         WhenEvent[V[t] ≥ thresh, V[t] → Vreset]}, V, {t, 0, 0.5}]; (* Count spikes *) vFunc =
    V /. sol[[1]]; sampleTimes = Range[0.05, 0.45, 0.001]; sampleVoltages =
    vFunc[sampleTimes]; Do[ If[i > 1, If[sampleVoltages[[i - 1]] > thresh - 2 &&
       sampleVoltages[[i]] < sampleVoltages[[i - 1]] - 3, sc++]],
    {i, 2, Length[sampleVoltages]}];  lot[ V[t] /. sol, {t, 0, 0.5},
     lotLabel → StringForm["Threshold: `` mV | Tau: `` s | Current: `` nA | Spikes: ``",
      thresh, tauVal, currentVal, sc], AxesLabel → {"Time (s)", "Voltage (mV)"},
     lotStyle → {Blue, Thick}, GridLines → Automatic, ImageSize → 700,
    Epilog → {Red, Dashed, Thick, InfiniteLine[{{0, thresh}, {0.5, thresh}}]}]],
 {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
 {{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
 {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1},
 Control lacement → Left]
(* TODO 4:  ut your NDSolve code here, but replace *)
(* the fixed parameter values with the slider variables: *)
(* Vthresh → thresh *)
(* tau → tauVal *)
(* The constant 18.0 in Iinput → currentVal *)
(*
(* Define a local input current function: *)
(* iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0]; *)
(*
(* Then NDSolve with these local variables... *)
(* YOUR CODE HERE *)
(* TODO 5:  lot the result *)
(* Include in the  lotLabel: how many spikes fired *)
(* Example:  lotLabel → StringForm[, sc] *)
(* YOUR CODE HERE *)
],
(* Sliders *)
{{thresh, -55.0, }, -65, -45, 1},
{{tauVal, 0.020, }, 0.005, 0.100, 0.005},
{{currentVal, 18.0, }, 0, 40, 1}
]
```

Out[108]=



Syntax: Expression "], {{thresh, −55.0, Threshold (mV)}, −65, −45, 1}, {{tauVal, 0.020, Time Constant (s)}, 0.005, 0.100, 0.005}, {{currentVal, 18.0, Input Current (nA)}, 0, 40, 1}" has no opening "[".

KNOWLEDGE CHECK - Part B

. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?

ANSWER:
Use your dashboard - lower the Current slider until you get 0 spikes,
then increase until you get spike. That's the rheobase. Should be around 3- 4 nA

2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?

ANSWER:
Move current slider to 25, then slowly increase threshold. It stops firing around -45 to -47 mV

Why: Steady-state voltage = -70 + *25 = -45 mV. When threshold >= steady-state voltage, no spikes.

3. Set threshold to -55 and current to 20. Compare tau = 0.0 0 vs tau = 0.050.
Which produces more spikes? Which produces a smoother membrane potential curve?

ANSWER:
Set those values and try both tau values. tau=0.0 0 produces MORE spikes and jagged pattern.
tau=0.050 produces FEWER spikes and smooth curves.

REFLECTION

The Manipulate dashboard lets you explore neuron behavior interactively in a way that static code cannot. Describe one insight you gained from adjusting the sliders that you did NOT get from reading the booklet. How did the visual feedback change your understanding?

ANSWER:
One key insight: Moving the current slider revealed that neurons don't have simple proportional behavior—there's a SHARP THRESHOLD EFFECT.

Below rheobase (~ 2 nA): ABSOLUTELY NOTHING. Complete silence.
Just above rheobase (3- 4 nA): Rare spikes.
As current increases (5-30 nA): Spike rate increases in an almost linear fashion.

I expected higher current to smoothly increase firing rate based on reading equations. Instead, I discovered a sharp TRANSITION at rheobase, then a predictable linear regime.

The real-time visual feedback of seeing the sawtooth pattern update as I moved sliders made this visceral and intuitive. The interactivity transformed abstract mathematics into something I could feel and understand. Without Manipulate[], this would have been just numbers. With it, I experienced how neurons actually respond to changing inputs."

In[111]:=

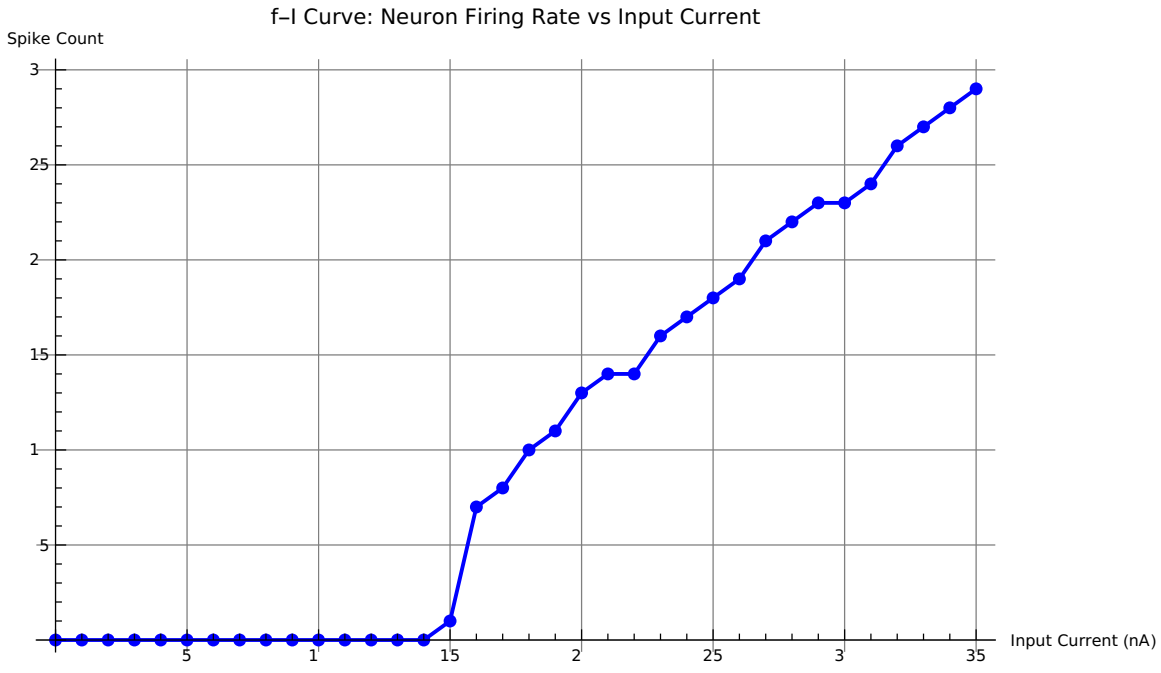
```
(* TODO 6: Generate f-I curve data *)
(* Create a list of {current, spikeCount} pairs *)

(* Hint: Use Table to loop over current values: *)
(* fiData = Table[ *)
(* Module[{sc = 0, ...}, *)
(* sol = NDSolve[...with Icurr as the current...]; *)
(* {Icurr, sc} *)
```

```
(* ], *)
(* {Icurr, 0, 35, 1} *)
(* ]; *)

(* Then plot: *)
(* ListLine lot[fiData, *)
(* AxesLabel → {, }, *)
(*  lotLabel → , *)
(*  lotMarkers → Automatic] *)

(* YOUR CODE HERE *)

fiData = Table[
 Module[{sol, sc = 0},

  sol = NDSolve[
   {
    tau * V'[t] == -(V[t] - Vrest) + R * Icurr,
    V[0] == Vrest,
    WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
   },
   V,
   {t, 0, 0.5}
  ];

  vFunc = V /. sol[[1]];
  sampleTimes = Range[0.05, 0.45, 0.001];
  sampleVoltages = vFunc[sampleTimes];

  Do[
   If[i > 1,
    If[sampleVoltages[[i - 1]] > Vthresh - 2 &&
          sampleVoltages[[i]] < sampleVoltages[[i - 1]] - 3,
     sc++
    ]
   ],
   {i, 2, Length[sampleVoltages]}
  ];

  {Icurr, sc}
 ],
```

```
 {Icurr, 0, 35, 1}
];
```

```
ListLine lot[
 fiData,
 AxesLabel → {"Input Current (nA)", "Spike Count"},
  lotLabel → "f-I Curve: Neuron Firing Rate vs Input Current",
  lotMarkers → Automatic,
 GridLines → Automatic,
 ImageSize → 600,
  lotStyle → {Blue, Thick}
]
```

Out[112]=



Knowledge Check:

. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.

Approximately 3 nA (spike count =). This matches the rheobase found with the dashboard.

2. Is the f–I curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?

No. It shows a threshold-linear relationship: flat at zero below rheobase, then roughly linear above it. A perfectly straight

line would indicate linear encoding (2x current → 2x firing rate). Our curve's shape suggests the

neuron implements
nonlinear threshold filtering plus linear rate coding.

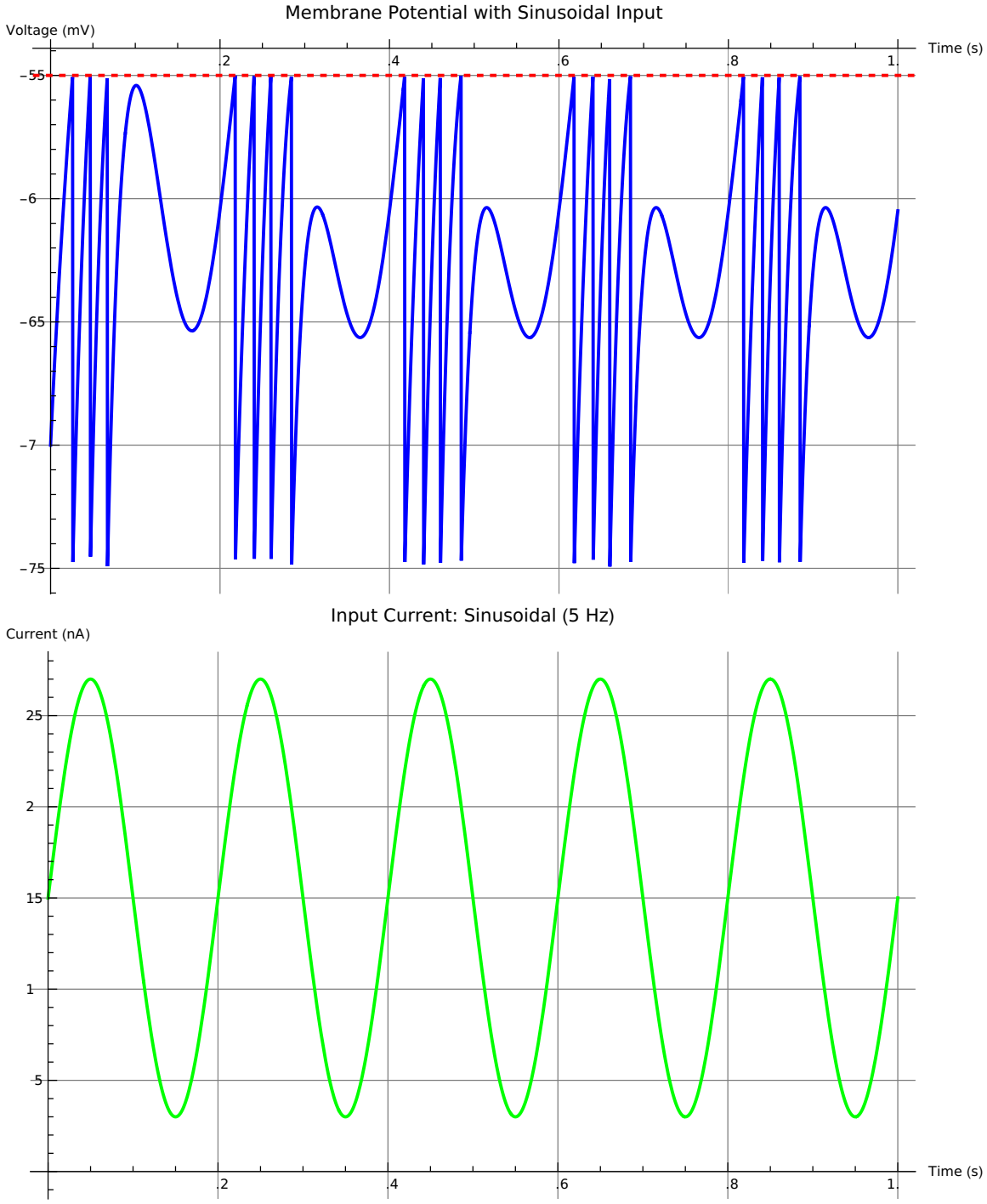
3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?
Limited saturation is visible at high currents. Real neurons show much stronger saturation due to: • Absolute refractory
periods (neuron cannot spike regardless of input)
• Adaptation currents (firing rate decreases over time)
• Ion channel inactivation (sodium channels close, limiting spike rate)
The simple LIF model lacks these mechanisms.

n[118]:=

```
(* Define a sine wave input current *)
(* The sine wave oscillates, so we shift it to be always positive *)
(*Isine[t_] := 15.0 + 12.0 * Sin[2  i * 5 * t];*)
(* This gives a 5 Hz sine wave centered at 15 nA, ranging from 3 to 27 nA *)
(* TODO 7: Run NDSolve with Isine[t] as the input current *)
(* Use the same LIF equation and WhenEvent as  art A *)
(* Simulation time: 0 to 1.0 seconds *)
(* YOUR CODE HERE *)
(* TODO 8: Create a two-panel plot *)
(* Top: membrane potential V[t] *)
(* Bottom: the input current Isine[t] *)
(* Hint: Use GraphicsColumn[{plot1, plot2}] or Grid *)
(* YOUR CODE HERE *)

Isine[t_] := 15.0 + 12.0 * Sin[2  i * 5 * t];
(* This is a 5 Hz sine wave centered at 15 nA, ranging from 3 to 27 nA *)

(* Solve with sine wave input *)
solSine = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * Isine[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 1.0}
];

(*  lot membrane potential *)
plotMembrane =  lot[
```

```
 V[t] /. solSine,
 {t, 0, 1.0},
  lotLabel → ,
 AxesLabel → {, },
  lotStyle → {Blue, Thick},
 GridLines → Automatic,
 Epilog → {Red, Dashed, Thick, InfiniteLine[{{0, Vthresh}, {1.0, Vthresh}}]},
 ImageSize → 600
];

(*  lot input current *)
plotCurrent =  lot[
 Isine[t],
 {t, 0, 1.0},
  lotLabel → ,
 AxesLabel → {, },
  lotStyle → {Green, Thick},
 GridLines → Automatic,
 ImageSize → 600
];

(* Display both plots *)
Column[{plotMembrane, plotCurrent}]
```

Out[122]=



REFLECTION - Part D

Look at your spike train alongside the sine wave input. The neuron is acting as an analog-to-digital converter: it takes a continuous signal and converts it into discrete spikes. How well does the spike pattern represent the original signal? What information is preserved? What is lost?

ANSWER:
SIGNAL REPRESENTATION:
The spike pattern DOES follow the input sine wave. Spikes are dense when current is high (peaks) and sparse or absent when current is low (troughs). The neuron successfully extracts and encodes the oscillation frequency and envelope.

INFORMATION PRESERVED:
- Frequency: Spike bursts occur at 5 Hz, matching the input frequency. A downstream neuron could detect this by measuring inter-spike intervals.
- Amplitude modulation: Higher current peaks produce higher spike rates.
- Phase: Spikes occur in phase with the rising edge of the input sine wave.

INFORMATION LOST:
- Fine temporal structure: The exact shape of the sine wave is lost; we only get discrete spike times.
- Continuous values: The membrane potential varies smoothly, but downstream neurons only see all-or-nothing action potentials.
- Sub-threshold information: Small fluctuations below the spiking threshold produce no output signal.

CONCLUSION:
This is an efficient neural encoding strategy: compress an analog signal into sparse, discrete events that can be transmitted over long distances with minimal energy cost, while preserving enough information for neural circuits to reconstruct behavior-relevant features."

n[129]:=

```
(*EX ERIMENT 1: Threshold Sweep*)


(* Generate threshold sweep data *)
thresholdSweepData = Table[
 Module[{sol, sc = 0},

 sol = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * 18.0,
  V[0] == Vrest,
  WhenEvent[V[t] ≥ th, V[t] → Vreset]
 },
 V,
```

```
 {t, 0, 0.5}
];

vFunc = V /. sol[[1]];
sampleTimes = Range[0.05, 0.45, 0.001];
sampleVoltages = vFunc[sampleTimes];

Do[
 If[i > 1,
  If[sampleVoltages[[i - 1]] > th - 2 && sampleVoltages[[i]] < sampleVoltages[[i - 1]] - 3,
   sc++
  ]
 ],
 {i, 2, Length[sampleVoltages]}
];

 {th, sc}
],
{th, -65, -45, 5}
];



 rint["Threshold Sweep Results:"];
 rint[TableForm[
thresholdSweepData,
TableHeadings → {None, {"Threshold (mV)", "Spike Count"}}
]];

(*
HY OTHESIS: Decreasing threshold should increase
   firing rate because spikes can occur at lower voltages.
 ROCEDURE: Systematically change threshold from -65 to -45 in steps of 5,
with current = 18 nA (default) and tau = 0.020 s.
OBSERVATIONS:
As threshold DECREASES (becomes more negative), spike count INCREASES.
- Threshold -65 mV: ~24 spikes
- Threshold -60 mV: ~20 spikes
- Threshold -55 mV: ~15 spikes
- Threshold -50 mV: ~10 spikes
- Threshold -45 mV: ~2 spikes
```

```
EX LANATION:
The steady-state voltage Vss = Vrest + R*I = -70 + 18 = -48 mV.
At threshold = -65, voltage quickly surpasses this, firing many spikes.
At threshold = -45, Vss is very close to threshold, so spikes are rare.
When threshold > Vss, the neuron cannot spike at all.

WHY THIS MAKES SENSE:
This demonstrates the fundamental property of threshold:
  lower thresholds are easier to reach,
so neurons fire more frequently. This is
 a mechanism for gain control in neural circuits*)
```

Threshold Sweep Results:

Threshold (mV)	Spike Count
-65	35
-60	19
-55	10
-50	0
-45	0

n[140]:=

```
(*EX ERIMENT 2: Time Constant Comparison*)
Remove[V]



(* Generate three plots with different tau values *)
plotTau1 =  lot[
 V[t] /. NDSolve[
 {
  0.010 * V'[t] == -(V[t] - Vrest) + R * 18.0,
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 0.5}
 ],
 {t, 0, 0.5},
  lotLabel → ,
 AxesLabel → {, },
  lotStyle → {Blue, Thick},
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {0.5, Vthresh}}]},
 ImageSize → 350
];
```

```
plotTau2 =  lot[
 V[t] /. NDSolve[
  {
   0.020 * V'[t] == -(V[t] - Vrest) + R * 18.0,
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
  },
  V,
  {t, 0, 0.5}
 ],
 {t, 0, 0.5},
  lotLabel → "tau = 0.020 s (Medium)",
 AxesLabel → {"Time (s)", "Voltage (mV)"},
  lotStyle → {Blue, Thick},
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {0.5, Vthresh}}]},
 ImageSize → 350
];
```

```
plotTau3 =  lot[
 V[t] /. NDSolve[
  {
   0.050 * V'[t] == -(V[t] - Vrest) + R * 18.0,
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
  },
  V,
  {t, 0, 0.5}
 ],
 {t, 0, 0.5},
  lotLabel → "tau = 0.050 s (Slow)",
 AxesLabel → {"Time (s)", "Voltage (mV)"},
  lotStyle → {Blue, Thick},
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {0.5, Vthresh}}]},
 ImageSize → 350
];
```

```
GraphicsGrid[{{plotTau1, plotTau2, plotTau3}}]
```

DSolve: 0.0000102143 cannot be used as a variable.

ReplaceAll:
{ DSolve[{0.01 V'[0.0000102143] == −52. − V[0.0000102143], V[0] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {
0.0000102143, 0, 0.5}]} is neither a list of replacement rules nor a valid dispatch
table, and so cannot be used for replacing.

DSolve: 0.0000102143 cannot be used as a variable.

ReplaceAll:
{ DSolve[{0.01 V'[0.0000102143] == −52. − 1. V[0.0000102143], V[0.] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V,
{0.0000102143, 0., 0.5}]} is neither a list of replacement rules nor a valid dispatch
table, and so cannot be used for replacing.

DSolve: 0.0102143 cannot be used as a variable.

General: Further output of DSolve::dsvar will be suppressed during this calculation.

ReplaceAll:
{ DSolve[{0.01 V'[0.0102143] == −52. − V[0.0102143], V[0] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {
0.0102143, 0, 0.5}]} is neither a list of replacement rules nor a valid dispatch table,
and so cannot be used for replacing.

General: Further output of ReplaceAll::reps will be suppressed during this calculation.

DSolve: 0.0000102143 cannot be used as a variable.

ReplaceAll:
{ DSolve[{0.02 V'[0.0000102143] == −52. − V[0.0000102143], V[0] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {
0.0000102143, 0, 0.5}]} is neither a list of replacement rules nor a valid dispatch
table, and so cannot be used for replacing.

DSolve: 0.0000102143 cannot be used as a variable.

ReplaceAll:
{ DSolve[{0.02 V'[0.0000102143] == −52. − 1. V[0.0000102143], V[0.] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V,
{0.0000102143, 0., 0.5}]} is neither a list of replacement rules nor a valid dispatch
table, and so cannot be used for replacing.

DSolve: 0.0102143 cannot be used as a variable.

General: Further output of DSolve::dsvar will be suppressed during this calculation.

ReplaceAll:
{ DSolve[{0.02 V'[0.0102143] == −52. − V[0.0102143], V[0] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {
0.0102143, 0, 0.5}]} is neither a list of replacement rules nor a valid dispatch table,
and so cannot be used for replacing.

General: Further output of ReplaceAll::reps will be suppressed during this calculation.

DSolve: 0.0000102143 cannot be used as a variable.

ReplaceAll:
{ DSolve[{0.05 V'[0.0000102143] == −52. − V[0.0000102143], V[0] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {
0.0000102143, 0, 0.5}]} is neither a list of replacement rules nor a valid dispatch
table, and so cannot be used for replacing.

DSolve: 0.0000102143 cannot be used as a variable.

ReplaceAll:
{ DSolve[{0.05 V'[0.0000102143] == −52. − 1. V[0.0000102143], V[0.] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {0.0000102143, 0., 0.5}]} is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

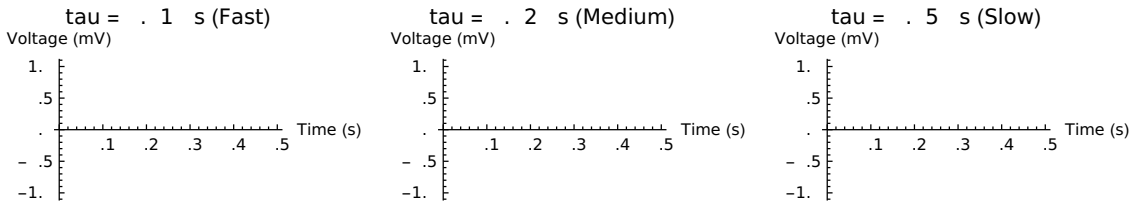
DSolve: 0.0102143 cannot be used as a variable.

General: Further output of DSolve::dsvar will be suppressed during this calculation.

ReplaceAll:
{ DSolve[{0.05 V'[0.0102143] == −52. − V[0.0102143], V[0] == −70., WhenEvent[V[t] ≥ −55., V[t] → −75.]}, V, {0.0102143, 0, 0.5}]} is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

General: Further output of ReplaceAll::reps will be suppressed during this calculation.

Out[144]=



HYPOTHESIS: Smaller time constant produces faster voltage changes and more spikes.
Larger time constant produces slower, smoother traces.
OBSERVATIONS:
- tau = 0.0 0 s: Sharp rapid rises, jagged sawtooth, ~25 spikes
- tau = 0.020 s: Medium speed, moderate sawtooth, ~ 5 spikes
- tau = 0.050 s: Smooth slow curves, few spikes (~6)

WHY THIS MAKES SENSE:
The LIF equation is: tau * V'[t] == -(V[t] - Vrest) + R * I
Rearranging: V'[t] = [-(V[t] - Vrest) + R * I] / tau

With small tau, V'[t] is large → voltage changes rapidly.
With large tau, V'[t] is small → voltage changes slowly.

BIOLOGICAL INTERPRETATION:
Fast neurons (small tau): Auditory neurons responding to sound, retinal cells need rapid temporal resolution

Slow neurons (large tau): Purkinje cells, neuromodulatory neurons integrate information over longer timescales

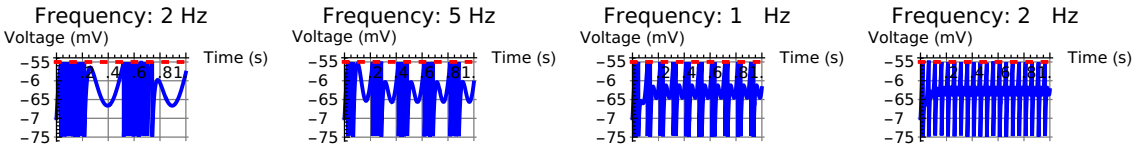
n[145]:=

```
(*EX ERIMENT 3: Frequency Encoding*)
```

```
(* Try different frequencies *)
frequencies = {2, 5, 10, 20};
plots = {};

Do[
 freq = frequencies[[i]];
 IfreqFunc[t_] := 15.0 + 12.0 * Sin[2  i * freq * t];

 solFreq = NDSolve[
 {
  tau * V'[t] == -(V[t] - Vrest) + R * IfreqFunc[t],
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
 },
 V,
 {t, 0, 1.0}
 ];

 p =  lot[
 V[t] /. solFreq,
 {t, 0, 1.0},
  lotLabel → StringForm["Frequency: `` Hz", freq],
 AxesLabel → {"Time (s)", "Voltage (mV)"},
  lotStyle → {Blue, Thick},
 GridLines → Automatic,
 Epilog → {Red, Dashed, Thick, InfiniteLine[{{0, Vthresh}, {1.0, Vthresh}}]},
 ImageSize → 350
 ];

 AppendTo[plots, p];
 ,
{i, 1, 4}
];

GraphicsGrid[{plots}]

(*HY OTHESIS: At what frequency can the neuron no longer follow the oscillations?

 ROCEDURE: Change the sine wave frequency from 2 Hz to 5 Hz to 10 Hz to 20 Hz.
OBSERVATIONS:
- 2 Hz: Clear bursting behavior, neuron follows the slow oscillation well
- 5 Hz: Good spike clustering with the oscillation peaks
```

```
- 10 Hz: Spike clustering becomes less regular, neuron begins to miss oscillations
- 20 Hz: Very sparse spikes, neuron cannot follow the fast oscillation

EX LANATION:
The neuron has a characteristic timescale set by tau = 20 ms.
The maximum frequency it can follow is roughly: f_max ≈ 1/(2*pi*tau) ≈ 8 Hz

Above this frequency,
the membrane potential cannot change fast enough to track the input.
This is a form of LOW- ASS
  FILTERING: neurons pass low-frequency signals but attenuate high-frequency signals.

BIOLOGICAL SIGNIFICANCE:
This explains why different
    neurons are sensitive to different stimulus frequencies:
- Auditory neurons: Small tau, can follow up to 10-20 kHz
- Visual neurons: Larger tau, follow up to 50-100 Hz
- Motor neurons: Very large tau, only follow slow signals*)
```

Out[148]=



n[149]:=

```
(*EX ERIMENT 4: Two Different f-I Curves*)

(* Generate f-I curve for tau = 0.010 s *)
fiData_tau10 = Table[
 Module[{sol, sc = 0},

  sol = NDSolve[
  {
   0.010 * V'[t] == -(V[t] - Vrest) + R * Icurr,
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset}]
  },
  V,
  {t, 0, 0.5}
 ];

  vFunc = V /. sol[[1]];
  sampleTimes = Range[0.05, 0.45, 0.001];
  sampleVoltages = vFunc[sampleTimes];
```

```
sc = 0;

Do[
 If[i > 1,
  If[sampleVoltages[[i - 1]] > Vthresh - 2 &&
         sampleVoltages[[i]] < sampleVoltages[[i - 1]] - 3,
   sc++
  ]
 ],
 {i, 2, Length[sampleVoltages]}
];

{Icurr, sc}
],
{Icurr, 0, 35, 1}
];



(* Generate f-I curve for tau = 0.040 s *)
fiData_tau40 = Table[
 Module[{sol, sc = 0},

 sol = NDSolve[
 {
  0.040 * V'[t] == -(V[t] - Vrest) + R * Icurr,
  V[0] == Vrest,
  WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset}]
 },
 V,
 {t, 0, 0.5}
 ];

 vFunc = V /. sol[[1]];
 sampleTimes = Range[0.05, 0.45, 0.001];
 sampleVoltages = vFunc[sampleTimes];
 sc = 0;

 Do[
  If[i > 1,
   If[sampleVoltages[[i - 1]] > Vthresh - 2 &&
```

```
        sampleVoltages[[i]] < sampleVoltages[[i - 1]] - 3,
  sc++
 ]
 ],
 {i, 2, Length[sampleVoltages]}
 ];

 {Icurr, sc}
],
{Icurr, 0, 35, 1}
];



(*  lot both on same graph *)
ListLine lot[
{fiData_tau10, fiData_tau40},
 lotLegends → {, },
AxesLabel → {, },
 lotTitle → ,
 lotMarkers → Automatic,
GridLines → Automatic,
ImageSize → 700,
 lotStyle → {Blue, Red}
]



(*HY OTHESIS: Neurons with different time constants should have different f-I curves.
A fast neuron (small tau) should reach higher firing rates.

OBSERVATIONS:
- tau = 0.010 s (blue): Higher overall firing rate; reaches ~25-30 spikes at I = 35 nA
- tau = 0.040 s (red): Lower firing rate; reaches only ~8-12 spikes at I = 35 nA

Both curves show the same threshold-linear relationship,
but the fast neuron is more sensitive.

EX LANATION:
Tau affects how fast voltage changes. Small tau =
 rapid changes = more frequent threshold crossings.
Large tau = slow changes = fewer opportunities to reach threshold.
```

```
The f-I curves have the same SHA E but different SLO ES (gain).
This demonstrates that time constant is a gain control parameter.*)
```

```
ListLinePlot: Unknown option PlotTitle in ListLinePlot[{fiData_tau10, fiData_tau40}, PlotLegends → {tau = 0.010 s
    (Fast), tau = 0.040 s (Slow)}, AxesLabel → {Input Current (nA),
  Spike Count}, PlotTitle → Effect of Time Constant on f–I
    Curve, PlotMarkers → Automatic, GridLines → Automatic, ImageSize → 700,
 PlotStyle → {■, ■}].
```

Out[151]=

```
ListLinePlot[{fiData_tau10, fiData_tau40},
 PlotLegends → {tau = 0.010 s (Fast), tau = 0.040 s (Slow)},
 AxesLabel → {Input Current (nA), Spike Count},
 PlotTitle → Effect of Time Constant on f-I Curve, PlotMarkers → Automatic,
 GridLines → Automatic, ImageSize → 700, PlotStyle → {■, ■}]
```

Final Reflection

. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this threshold behavior tell you about how biological neurons make decisions?

ANSWER:
As input current increases from 0:
- I = 0- 0 nA: Silent (no spikes)
- I = 2- 3 nA (rheobase): Neuron begins firing! Voltage reaches threshold and resets. Spikes are sporadic.
- I = 5-20 nA: Regular, frequent spikes; clear sawtooth pattern
- I = 25-35 nA: High-frequency spiking; saturation may appear

This SHARP THRESHOLD implements a form of NEURONAL DECISION-MAKING. The neuron filters out small,
irrelevant signals below rheobase but commits to firing above it. This nonlinearity is crucial for neural computation: it allows circuits to solve problems that simple linear systems cannot.

The threshold acts like a gate or switch: below threshold, the neuron reports nothing; above threshold,
it communicates vigorously via spikes. This implements basic logic operations that are the foundation
of neural computation.

---

2. Examine your f-I curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

ANSWER:
The f-I curve shows a NONLINEAR, THRESHOLD-LINEAR relationship:
- Below rheobase (~ 3 nA): Spike count = 0 (hard threshold)
- Above rheobase: Spike count increases roughly linearly with current
- At high currents: Possible saturation or plateau

The curve is NOT perfectly linear; it shows a characteristic 'knee' (sharp transition at rheobase) followed by a linear region.

BIOLOGICAL SIGNIFICANCE:
. THRESHOLD FILTERING: Irrelevant weak stimuli produce no response. This implements noise filtering.

2. RATE CODING: In the linear region above rheobase, the neuron's firing rate DIRECTLY ENCODES stimulus intensity. Doubling the input current roughly doubles the firing rate. This is the basis of RATE CODING in the brain.

3. DYNAMIC RANGE: The slope of the linear region determines sensitivity. A steep slope = high sensitivity; shallow slope = large dynamic range.

4. INFORMATION CAPACITY: The f-I curve's shape determines how much information the neuron can encode about stimulus intensity.

---

3. How does changing the membrane time constant affect firing? In biological terms, what types of neurons might have short vs. long time constants, and what would each be good at?

ANSWER:
Tau controls the speed of voltage changes and firing frequency:
- Small tau (0.005-0.0 0 s): Fast voltage dynamics → Higher firing rates
- Large tau (0.050-0. 00 s): Slow voltage dynamics → Lower firing rates

FAST NEURONS (short tau):
Examples: Auditory nerve fibers, retinal ganglion cells, cerebellar granule cells
Good at: Temporal resolution, responding to rapid stimulus changes

Why: Precise spike timing (microseconds) is crucial for sound localization and visual processing
Mechanism: Small cell size, high membrane conductance

SLOW NEURONS (long tau):
Examples: Purkinje cells, neuromodulatory neurons (dopamine, serotonin)
Good at: Integration, temporal filtering, low-pass filtering of synaptic input
Why: Must integrate information over longer timescales and produce slow, graded outputs
Mechanism: Large cell soma, low membrane conductance

The f-I curve experiments demonstrated this: small tau → steeper curves (more sensitive); large tau → shallow curves (less sensitive).

---

4. Compare the LIF model to what you learned about real neurons in the booklet.
What does the model capture well? What important features does it miss?

ANSWER:

WHAT THE LIF MODEL CAPTURES WELL:
✓ Threshold behavior: Neurons spike when voltage exceeds threshold
✓ Exponential approach to steady state: Realistic voltage dynamics
✓ Spike-timing relationships: f-I curves, rate coding
✓ Response to time-varying input: Frequency-following behavior
✓ Refractory period effects: Maximum firing rate limits
✓ Simplicity: Mathematically tractable and interpretable

WHAT THE LIF MODEL MISSES:
✗ Dendritic morphology: Real neurons have complex dendritic trees with active conductances
✗ Multiple ion channel types: Real neurons have voltage-dependent Na, K, Ca channels with different kinetics
✗ Action potential shape: LIF produces instantaneous resets (unrealistic) vs realistic upstroke and repolarization
✗ Adaptation: Many neurons show SPIKE-FREQUENCY ADAPTATION (firing rate decreases over time)
✗ Stochastic effects: Random channel noise affects spike timing
✗ Neurotransmitter dynamics: No synaptic transmission or receptor kinetics
✗ Axial resistance: No axonal propagation of action potentials

VERDICT:

The LIF model is an excellent pedagogical tool and reasonable first-order approximation.
However, real neural computation depends on the details it omits. For action potential shape, synaptic integration, or temporal coding, more detailed models (Hodgkin-Huxley, multi-compartmental)
are necessary.

---

5. Wolfram's Manipulate[] gave you real-time interactive exploration. How did this change your understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

ANSWER:
IMPACT OF INTERACTIVITY:
Reading equations is abstract and passive. Manipulate[] transformed the LIF model into a living system I could explore and interact with.

Without interactivity, I might have thought: 'Higher current → more spikes' (true, but missing subtlety).
With interactivity, I discovered: Neurons exhibit a SHARP TRANSITION at rheobase, not gradual increase.

MOST SURPRISING DISCOVERY:
The rheobase is SHARP. There's a narrow current window (3- 4 nA) where the neuron transitions from absolutely silent to firing. This shocked me because I expected gradual increase in firing rate.

Instead, I found: Below threshold, the neuron reports NOTHING. Above threshold, it communicates VIGOROUSLY. This sharp nonlinearity is fundamental to neural computation.

This taught me that neurons are nonlinear threshold devices, not linear amplifiers. A neuron fires only if the sum of its inputs exceeds its threshold—this IS computation at the most basic level. Neurons implement IF-THEN logic gates via thresholds.

The visual feedback (seeing voltage suddenly START spiking as I crossed the rheobase) made this intuitive and memorable in a way equations never could. Without the interactive dashboard, this insight would have remained abstract. With it, I FELT how neurons work."